ADVANCED HMS

1 MONTH INTENSIVE BUILD PLAN

From Good Student Project → Advanced Portfolio-Worthy HMS

Total Duration	30 Days (4 Weeks + 2 Buffer Days)
Daily Commitment	4–6 Hours per Day
What You Have Now	Phase 1 Complete — Core HMS Working ■
What You'll Have	AI-Powered Advanced HMS with Security & Real-time Features
Unique Features	Drug Checker + No-Show ML + Triage + WhatsApp + Audit Trail
Tech Added	Redis, Celery, PostgreSQL, Claude API, scikit-learn, WebSockets
Deployment	Docker + Cloud (Day 29-30)
Portfolio Value	Senior Developer Level Project

■■ HONEST REALITY CHECK

1 month is tight. We CANNOT build everything. So we build the RIGHT things —
the features that make it UNIQUE and IMPRESSIVE. Quality over quantity.
We skip: Mobile app, FHIR, ICD coding, Telemedicine (save for later).
We focus on: Security + 3 AI features + WhatsApp + Real-time + Deploy.

THE 30-DAY MASTER PLAN

Week	Days	Focus	Key Deliverable
Week 1	Day 1–7	Security & Infrastructure	PostgreSQL + Redis + Audit Trail + 2FA
Week 2	Day 8–14	AI Feature 1 & 2	Drug Interaction Checker + No-Show Predictor
Week 3	Day 15–21	AI Feature 3 + WhatsApp	AI Triage System + WhatsApp Notifications
Week 4	Day 22–28	Reports + Real-time + Polish	Admin Reports + WebSockets + Bug fixes
Buffer	Day 29–30	Deploy	Docker + Cloud Deployment + README

Daily Schedule Recommendation

Time Block	Activity
Hour 1	Review yesterday's work, fix any bugs
Hours 2–3	Main coding (new feature)
Hour 4	Testing + writing test cases
Hour 5	Documentation + Git commit
Hour 6 (if free)	Extra feature OR reading docs for tomorrow

Skip a day? Don't panic. The buffer days (29–30) are for catching up. Every feature has a MUST DO and NICE TO HAVE — always do MUST DO first.

WEEK 1 — SECURITY & PRODUCTION INFRASTRUCTURE

Day 1–7 | Make It Production-Grade Before Adding AI

WHY WEEK 1 IS SECURITY: You cannot add AI to a broken foundation. Week 1 makes your app trustworthy, scalable, and impressive to any developer who reviews your code.

DAY 1 — PostgreSQL Migration

Switch from SQLite to PostgreSQL. This single change makes your project look 10x more professional.

Tasks

- Install PostgreSQL locally
- Install psycopg2-binary
- Update settings.py DATABASE config
- Run migrations on PostgreSQL
- Verify all existing data works

```
pip install psycopg2-binary dj-database-url

# settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.getenv('DB_NAME', 'hmsdb'),
        'USER': os.getenv('DB_USER', 'hmsuser'),
        'PASSWORD': os.getenv('DB_PASSWORD', 'yourpassword'),
        'HOST': 'localhost',
        'PORT': '5432',
        'CONN_MAX_AGE': 60,
    }
}
```

New Model Fields to Add

```
# users/models.py - Add to PatientProfile:
from django.contrib.postgres.fields import ArrayField

allergies = ArrayField(
    models.CharField(max_length=100),
    blank=True, default=list,
    help_text='List of drug allergies'
)
medical_history = models.JSONField(default=dict, blank=True)
```

Time Estimate

- 4 hours

DAY 2 — Redis + Celery Setup

Make emails, PDFs, and notifications happen in the background. Currently everything blocks the user. With Celery, user gets instant response while tasks run async.

- Install Redis (Windows: use WSL or Redis Windows port)
- Install Celery + django-celery-beat
- Create celery.py in hms_project/
- Move all email sending to Celery tasks
- Test: book appointment → email sent in background

```
pip install celery redis django-celery-beat django-redis

# settings.py
CELERY_BROKER_URL = 'redis://localhost:6379/0'
CELERY_RESULT_BACKEND = 'redis://localhost:6379/0'

# Also add Redis cache:
CACHES = {
    'default': {
        'BACKEND': 'django_redis.cache.RedisCache',
        'LOCATION': 'redis://127.0.0.1:6379/1',
    }
}

# hms_project/celery.py
from celery import Celery
import os
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'hms_project.settings')
app = Celery('hms_project')
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks()
```

Celery Tasks to Create

Task File	Task Name	Triggered When
notifications/tasks.py	send_appointment_email	Appointment booked
notifications/tasks.py	send_lab_report_email	Lab report released
notifications/tasks.py	send_payment_receipt_email	Payment marked paid
appointments/tasks.py	send_appointment_reminder	Celery Beat — daily 9AM
lab/tasks.py	generate_lab_pdf	Lab result saved

Time Estimate

- 5 hours

DAY 3 — Complete Audit Trail

Every action logged with who did what, when, from which IP. Cannot be deleted. Makes your project HIPAA-aware. Extremely impressive in demos.

- Create audit app
- Create AuditLog model
- Create middleware to auto-log all requests
- Add audit decorators to sensitive views
- Create audit log viewer for admin

```
# audit/models.py
class AuditLog(models.Model):
    ACTION_CHOICES = [
        ('CREATE','Created'), ('VIEW','Viewed'),
        ('UPDATE','Updated'), ('DELETE','Deleted'),
        ('LOGIN','Login'), ('LOGOUT','Logout'),
        ('DOWNLOAD','Downloaded'), ('FAILED_LOGIN','Failed Login'),
    ]
    user        = models.ForeignKey('users.User', null=True, on_delete=models.SET_NULL)
    action      = models.CharField(max_length=15, choices=ACTION_CHOICES)
    model_name  = models.CharField(max_length=100)
    object_id   = models.IntegerField(null=True)
    object_repr = models.CharField(max_length=200)
```

```
    old_values  = models.JSONField(null=True)
    new_values  = models.JSONField(null=True)
    ip_address  = models.GenericIPAddressField(null=True)
    timestamp   = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['-timestamp']

# audit/utils.py
def log_action(user, action, model_name, obj_id,
               obj_repr, old=None, new=None, request=None):
    ip = request.META.get('REMOTE_ADDR') if request else None
    AuditLog.objects.create(
        user=user, action=action,
        model_name=model_name, object_id=obj_id,
        object_repr=obj_repr, old_values=old,
        new_values=new, ip_address=ip
    )
```

Time Estimate

- 5 hours

DAY 4 — Two-Factor Authentication (2FA)

Doctors and admins must verify identity with OTP. Protects patient data if password is stolen.

- Install django-otp
- Add OTP device to Doctor and Admin accounts
- Create 2FA setup page (QR code for Google Authenticator)
- Create OTP verification middleware
- Add 2FA status badge to admin dashboard

```
pip install django-otp qrcode django-allauth

# Add to INSTALLED_APPS:
'django_otp', 'django_otp.plugins.otp_totp',

# users/models.py - Add to User:
two_factor_enabled = models.BooleanField(default=False)

# Middleware: force 2FA for Doctor/Admin:
ROLES_REQUIRING_2FA = ['DOCTOR', 'ADMIN', 'LAB_TECHNICIAN']

# In login view — after password check:
if user.role in ROLES_REQUIRING_2FA and user.two_factor_enabled:
    request.session['pre_2fa_user'] = user.id
    return redirect('users:verify_2fa')
```

Time Estimate

- 5 hours

DAY 5 — Rate Limiting + Brute Force Protection

- Install django-axes (brute force lockout)
- Install django-ratelimit (API rate limiting)
- Configure: max 5 failed logins → lockout 1 hour
- Add rate limit to appointment booking API

- Create static/ folder (fix that warning!)

```
pip install django-axes django-ratelimit

# settings.py
AXES_FAILURE_LIMIT = 5
AXES_COOLOFF_TIME  = 1  # 1 hour
AXES_LOCKOUT_TEMPLATE = 'users/locked_out.html'

# Add to INSTALLED_APPS: 'axes'
# Add to MIDDLEWARE: 'axes.middleware.AxesMiddleware'

# Fix static folder warning:
import os
os.makedirs(BASE_DIR / 'static', exist_ok=True)
STATICFILES_DIRS = [BASE_DIR / 'static']
```

Time Estimate

- 3 hours — Use remaining time to fix any Week 1 bugs

DAY 6–7 — Reviews App + Dashboard Fixes

Build the reviews app (skipped earlier) and fix all dashboard stat counts. This completes Phase 1 properly.

- Create reviews app — Review model (patient, doctor, appointment, rating 1-5, comment)
- Only allow review after COMPLETED appointment
- One review per appointment (OneToOne with Appointment)
- Show average rating on doctor search page
- Fix ALL dashboard counts (patient, doctor, admin, lab tech)
- Fix notification bell count
- Add missing nav links for all roles

Reviews Model

```
# reviews/models.py
class Review(models.Model):
    patient     = models.ForeignKey('users.PatientProfile', on_delete=models.CASCADE)
    doctor      = models.ForeignKey('clinical.Doctor', on_delete=models.CASCADE,
                                    related_name='reviews')
    appointment = models.OneToOneField('appointments.Appointment',
                                       on_delete=models.CASCADE)
    rating      = models.IntegerField()  # 1-5
    comment     = models.TextField(blank=True)
    created_at  = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ['patient', 'appointment']
```

Week 1 Deliverable — What You Have After Day 7

- ■ PostgreSQL running
- ■ Celery + Redis async tasks
- ■ Audit trail logging every action
- ■ 2FA for doctors and admins
- ■ Brute force protection
- ■ Reviews app complete
- ■ All dashboards showing real data
- ■ All nav links working

WEEK 2 — AI FEATURES 1 & 2 — Drug Checker + No-Show Predictor

Day 8–14 | The Features That Make It Unique

This week is the HARDEST. But these 2 AI features are what makes your HMS stand out from EVERY other student project. Push through.

DAY 8–9 — Drug Interaction Checker

When doctor prescribes a medicine, system instantly checks if it interacts dangerously with patient's existing medications. Saves lives. No other student HMS has this.

Day 8 — Setup Database

- Create drug_checker app
- Create DrugInteraction model
- Create DrugAllergyAlert model
- Download FDA drug interaction data (free)
- Write management command to import drug data into DB

```
# drug_checker/models.py
class DrugInteraction(models.Model):
    SEVERITY = [('MINOR','Minor'),('MODERATE','Moderate'),
                ('MAJOR','Major'),('CONTRAINDICATED','Do Not Use Together')]
    drug_a      = models.CharField(max_length=200)  # Generic name lowercase
    drug_b      = models.CharField(max_length=200)
    severity    = models.CharField(max_length=20, choices=SEVERITY)
    description = models.TextField()
    management  = models.TextField()  # What doctor should do

class DrugAllergyAlert(models.Model):
    patient  = models.ForeignKey('users.PatientProfile', on_delete=models.CASCADE)
    drug     = models.CharField(max_length=200)
    reaction = models.TextField()
    severity = models.CharField(max_length=20)
```

Day 9 — API + Frontend Integration

- Create JSON API endpoint: /api/check-drug-interaction/
- JavaScript: call API every time doctor adds a medicine
- Show WARNING badge in prescription form
- Doctor must check 'I acknowledge this warning' before saving
- Log all acknowledged warnings in PrescriptionInteractionLog

```
# drug_checker/views.py
from rapidfuzz import process, fuzz

def check_interaction(request):
    drug_a = request.GET.get('drug_a', '').lower()
    drug_b = request.GET.get('drug_b', '').lower()
    # Fuzzy match drug names
    interactions = DrugInteraction.objects.filter(
        Q(drug_a__icontains=drug_a) | Q(drug_b__icontains=drug_a)
    ).filter(
        Q(drug_a__icontains=drug_b) | Q(drug_b__icontains=drug_b)
    )
    if interactions.exists():
        worst = interactions.order_by('-severity').first()
        return JsonResponse({'found': True, 'severity': worst.severity,
                             'description': worst.description,
```

```
                            'management': worst.management})
    return JsonResponse({'found': False})

# Install: pip install rapidfuzz
```

Time Estimate

- Day 8: 5 hours setup | Day 9: 5 hours integration

DAY 10–11 — No-Show Predictor ML Model

ML model predicts which patients will not show up. High-risk patients get auto WhatsApp reminder (we'll add WhatsApp in Week 3). Solves the 40% no-show problem.

Day 10 — Build & Train Model

- Create ml_noshow app
- Build dataset from existing appointments
- Train XGBoost classifier
- Save model as .pkl file
- Create NoShowPrediction model

```
# ml_noshow/models.py
class NoShowPrediction(models.Model):
    appointment  = models.OneToOneField('appointments.Appointment',
                                        on_delete=models.CASCADE)
    probability  = models.DecimalField(max_digits=5, decimal_places=2)
    risk_level   = models.CharField(max_length=6,
                      choices=[('LOW','Low'),('MEDIUM','Medium'),('HIGH','High')])
    action_taken = models.CharField(max_length=20, default='NONE')
    actual_shown = models.BooleanField(null=True)  # Ground truth
    created_at   = models.DateTimeField(auto_now_add=True)

# pip install xgboost scikit-learn joblib pandas numpy
```

```
# ml_noshow/train.py
import pandas as pd
from xgboost import XGBClassifier
import joblib

# Features to use (even with small dataset):
# day_of_week, hour_of_day, days_in_advance, patient_age
# previous_noshow_count, previous_appointment_count
# payment_status (unpaid = higher no-show risk)

model = XGBClassifier(n_estimators=100, max_depth=4)
model.fit(X_train, y_train)
joblib.dump(model, 'ml_noshow/noshow_model.pkl')
print('Model saved!')
```

Day 11 — Integrate Into Appointment Booking

- Run prediction as Celery task after appointment created
- Show risk badge on appointment list (doctor view)
- Admin can see no-show risk dashboard
- Celery Beat: daily 8AM check upcoming appointments

Time Estimate

- Day 10: 6 hours | Day 11: 4 hours

DAY 12–13 — Heart Attack Risk Predictor

Patient fills 13-field form with health data. ML model predicts heart attack risk with probability percentage. Saves as assessment record.

```
# ml_heart/models.py
class HeartRiskAssessment(models.Model):
    patient         = models.ForeignKey('users.PatientProfile', on_delete=models.CASCADE)
    age             = models.IntegerField()
    sex             = models.IntegerField()
    chest_pain_type = models.IntegerField()
    resting_bp      = models.IntegerField()
    cholesterol     = models.IntegerField()
    fasting_bs      = models.IntegerField()
    resting_ecg     = models.IntegerField()
    max_heart_rate  = models.IntegerField()
    exercise_angina = models.IntegerField()
    st_depression   = models.DecimalField(max_digits=4,decimal_places=1)
    st_slope        = models.IntegerField()
    major_vessels   = models.IntegerField()
    thalassemia     = models.IntegerField()
    risk_score      = models.DecimalField(max_digits=5, decimal_places=2)
    risk_level      = models.CharField(max_length=8)  # LOW/MEDIUM/HIGH
    assessed_at     = models.DateTimeField(auto_now_add=True)

# Algorithm: RandomForest + XGBoost ensemble
# Dataset: UCI Heart Disease (download free)
# Accuracy typically 85%+
```

Time Estimate

- Day 12: Train model + model (5h) | Day 13: Form + template + test (5h)

DAY 14 — Week 2 Buffer + Testing

- Test all drug interaction checks with real examples
- Fix any ML model issues
- Verify Celery tasks running correctly
- Write unit tests for critical functions
- Git commit with proper messages

Week 2 Deliverable

- ■ Drug interaction checker working in prescription form
- ■ Warning shown for dangerous combinations
- ■ No-show ML model predicting risk on booking
- ■ Heart attack predictor accessible from patient dashboard
- ■ All async tasks running in background via Celery

WEEK 3 — AI TRIAGE + WHATSAPP + AI REPORT READER

Day 15–21 | The Features Patients Actually See & Love

Week 3 adds the features that patients interact with directly. WhatsApp notifications will immediately impress any demo audience in Nepal.

DAY 15–16 — AI Triage System

Patient fills symptom form. Claude AI analyzes symptoms and assigns P1-P4 priority. Critical patients flagged for immediate attention. This is the most impressive AI demo.

```
# triage/models.py
class TriageAssessment(models.Model):
    PRIORITY = [('P1','CRITICAL'),('P2','URGENT'),
                ('P3','SEMI_URGENT'),('P4','NON_URGENT')]
    patient         = models.ForeignKey('users.PatientProfile', on_delete=models.CASCADE)
    priority        = models.CharField(max_length=3, choices=PRIORITY)
    chief_complaint = models.TextField()
    symptoms        = models.JSONField()
    pain_scale      = models.IntegerField(null=True)  # 0-10
    ai_reasoning    = models.TextField()
    ai_confidence   = models.DecimalField(max_digits=4, decimal_places=2)
    nurse_override  = models.CharField(max_length=3, null=True, blank=True)
    assessed_at     = models.DateTimeField(auto_now_add=True)
```

```
# triage/views.py — Uses Claude API
import anthropic

def triage_patient(request):
    if request.method == 'POST':
        client = anthropic.Anthropic(api_key=settings.ANTHROPIC_API_KEY)
        prompt = f"""
        Patient: {age} year old {gender}
        Complaint: {complaint}
        Symptoms: {symptoms}
        Pain: {pain}/10

        Assign triage priority. Respond ONLY in JSON:
        {{"priority":"P1|P2|P3|P4",
          "reasoning":"brief clinical reason",
          "confidence":0.95,
          "immediate_actions":["action1"]}}
        """
        response = client.messages.create(
            model='claude-sonnet-4-6',
            max_tokens=500,
            messages=[{'role':'user','content':prompt}]
        )
        # Parse JSON, save, redirect to priority queue

# pip install anthropic
```

Time Estimate

- Day 15: Model + API (5h) | Day 16: Form + Template + Queue view (5h)

DAY 17–18 — WhatsApp Notifications (Twilio)

80% of Nepal patients use WhatsApp daily. This is the most visible feature in demos. Patients receive appointment confirmations, reminders, and report notifications on WhatsApp.

WhatsApp Messages to Send

Event	Message Template
Appointment Booked	Your appointment with Dr. {name} is confirmed for {date} at {time}. HMS Hospital.
24h Reminder	Reminder: Appointment tomorrow at {time} with Dr. {name}. Reply CONFIRM or CANCEL.
No-Show Risk High	We miss you! Your appointment is in 2 hours. Reply YES to confirm attendance.
Prescription Ready	Your prescription from Dr. {name} is ready. Download: {link}
Lab Report Ready	Your {test} report is ready. View and download: {link}
Discharge	Discharged successfully! Bill: Rs. {amount}. Download summary: {link}

```
pip install twilio

# notifications/whatsapp.py
from twilio.rest import Client
from django.conf import settings

def send_whatsapp(phone_number, message):
    """Send WhatsApp message via Twilio Sandbox."""
    client = Client(settings.TWILIO_ACCOUNT_SID, settings.TWILIO_AUTH_TOKEN)
    try:
        client.messages.create(
            from_='whatsapp:+14155238886',  # Twilio sandbox number
            body=message,
            to=f'whatsapp:+977{phone_number}'  # Nepal +977
        )
        return True
    except Exception as e:
        print(f'WhatsApp failed: {e}')
        return False

# settings.py
TWILIO_ACCOUNT_SID = os.getenv('TWILIO_ACCOUNT_SID')
TWILIO_AUTH_TOKEN = os.getenv('TWILIO_AUTH_TOKEN')

# Add WhatsApp send to existing notification triggers:
# In appointments/views.py after save:
send_whatsapp_task.delay(patient.phone,
    f'Appointment confirmed with Dr. {doctor.name} on {date}')
```

Celery Beat — Appointment Reminders

```
# appointments/tasks.py
from celery import shared_task

@shared_task
def send_daily_reminders():
    """Run every day at 9 AM — remind tomorrow's patients."""
    from datetime import date, timedelta
    tomorrow = date.today() + timedelta(days=1)
    appointments = Appointment.objects.filter(
        date=tomorrow, status__in=['PENDING','CONFIRMED']
    )
    for apt in appointments:
        send_whatsapp(
            apt.patient.phone,
            f'Reminder: Appointment tomorrow at {apt.start_time} with Dr.
{apt.doctor.user.username}'
        )

# Celery Beat schedule:
```

```
CELERY_BEAT_SCHEDULE = {
    'daily-reminders': {
        'task': 'appointments.tasks.send_daily_reminders',
        'schedule': crontab(hour=9, minute=0),
    },
}
```

Time Estimate

- Day 17: Setup Twilio + basic sending (4h) | Day 18: Connect to all events + Celery Beat (5h)

DAY 19–20 — AI Medical Report Reader

Patient uploads any hospital report. Claude API reads it and explains in simple language. Shows which values are abnormal and what to do next.

```
# ai_reader/models.py
class ReportAnalysis(models.Model):
    user             = models.ForeignKey('users.User', on_delete=models.CASCADE)
    original_file    = models.FileField(upload_to='ai_reports/')
    report_type      = models.CharField(max_length=10,
                           choices=[('LAB','Lab'),('XRAY','X-Ray'),
                                    ('MRI','MRI'),('OTHER','Other')])
    ai_summary       = models.TextField()
    abnormal_findings= models.JSONField(default=list)
    urgency_flag     = models.CharField(max_length=10,
                           choices=[('NORMAL','Normal'),('WATCH','Watch'),('URGENT','Urgent')])
    created_at       = models.DateTimeField(auto_now_add=True)

# View — analyze uploaded PDF with Claude:
import anthropic, base64

def analyze_report(request):
    if request.method == 'POST':
        f = request.FILES['report']
        data = base64.b64encode(f.read()).decode()
        client = anthropic.Anthropic(api_key=settings.ANTHROPIC_API_KEY)
        msg = client.messages.create(
            model='claude-sonnet-4-6',
            max_tokens=1500,
            messages=[{'role':'user','content':[
                {'type':'document',
                 'source':{'type':'base64','media_type':'application/pdf','data':data}},
                {'type':'text','text':ANALYSIS_PROMPT}
            ]}]
        )
        result = msg.content[0].text
        # Save + show to patient
```

Time Estimate

- Day 19: Model + view + Claude integration (5h) | Day 20: Template + styling + test (4h)

DAY 21 — Week 3 Buffer + Polish

- Test all WhatsApp messages end-to-end
- Test AI triage with 5 different symptom scenarios
- Test AI report reader with a real PDF
- Fix any template issues
- Commit all code

Week 3 Deliverable

- ■ AI triage prioritizing patients by symptom severity
- ■ WhatsApp messages sent on all major events
- ■ Daily appointment reminders auto-running via Celery Beat
- ■ AI report reader analyzing uploaded PDFs
- ■ Patients can check heart attack risk from dashboard

WEEK 4 — REPORTS + REAL-TIME + POLISH + DEPLOY

Day 22–30 | Make It Shine & Ship It

DAY 22–23 — Admin Reports & Analytics Dashboard

Admin sees the full hospital picture — revenue, appointments, popular doctors, lab stats. Export as PDF. This is what hospital management actually needs.

```
# reports/views.py
from django.db.models import Sum, Count, Avg
from datetime import date, timedelta

def admin_reports(request):
    today = date.today()
    month_start = today.replace(day=1)

    context = {
        # Revenue
        'total_revenue':
Payment.objects.filter(status='PAID').aggregate(Sum('amount'))['amount__sum'] or 0,
        'monthly_revenue': Payment.objects.filter(
            status='PAID', paid_at__gte=month_start
        ).aggregate(Sum('amount'))['amount__sum'] or 0,
        'revenue_by_type': Payment.objects.filter(status='PAID').values(
            'payment_type').annotate(total=Sum('amount')),

        # Appointments
        'total_appointments': Appointment.objects.count(),
        'todays_appointments': Appointment.objects.filter(date=today).count(),
        'noshow_rate': ...,  # Calculate from actual outcomes

        # Doctors
        'top_doctors': Doctor.objects.annotate(
            apt_count=Count('appointments')
        ).order_by('-apt_count')[:5],

        # Lab
        'popular_tests': TestTemplate.objects.annotate(
            booking_count=Count('bookings')
        ).order_by('-booking_count')[:5],
    }
```

Time Estimate

- Day 22: Views + data queries (5h) | Day 23: Template + charts (JavaScript Chart.js) + PDF export (4h)

DAY 24–25 — Real-Time Updates (Django Channels)

Nurses and doctors see live updates without refreshing. Vital alerts, triage queue changes, new admissions — all update in real-time via WebSockets.

```
pip install channels channels-redis daphne

# 3 Real-time features to build:

# 1. Live notification bell (already polling — upgrade to WebSocket)
# 2. Live triage queue — admin sees patients in real-time
# 3. Vital sign alert — nurse sees alert pop up when MEWS > 5

# hms_project/asgi.py
```

```
from channels.routing import ProtocolTypeRouter, URLRouter
import notifications.routing

application = ProtocolTypeRouter({
    'http': django_asgi_app,
    'websocket': URLRouter(notifications.routing.websocket_urlpatterns)
})

# notifications/consumers.py
from channels.generic.websocket import WebsocketConsumer
import json

class NotificationConsumer(WebsocketConsumer):
    def connect(self):
        self.accept()
        self.send(json.dumps({'type':'connected'}))

    def send_notification(self, event):
        self.send(json.dumps(event['data']))
```

Time Estimate

- Day 24: Setup channels + basic WebSocket (5h) | Day 25: Connect to notifications + triage queue (5h)

DAY 26–27 — Full Polish Pass

The difference between a student project and a portfolio project is polish. Go through every single page and make it look professional.

Area	What To Fix
Navigation	Role-specific nav menus, active page highlighting, mobile responsive
Dashboards	All counts correct, loading states, empty states with helpful messages
Forms	Validation messages, success messages, loading buttons
Tables	Pagination on all lists, search/filter on key pages
PDFs	Test all 5 PDF types — prescriptions, receipts, lab reports, admission
Error pages	Custom 404, 500 pages with HMS branding
Mobile view	Test all pages on mobile size in Chrome DevTools
Performance	Add @cache_page to heavy views, optimize slow queries
README.md	Write comprehensive README with screenshots for GitHub

Time Estimate

- 2 full days — do not rush this

DAY 28 — Docker Setup

```
# docker-compose.yml
version: '3.8'
services:
  web:
    build: .
    command: gunicorn hms_project.wsgi:application --bind 0.0.0.0:8000 --workers 3
    volumes:
      - .:/app
    ports:
      - '8000:8000'
```

```
    env_file: .env
    depends_on: [db, redis]

  db:
    image: postgres:15
    volumes: [postgres_data:/var/lib/postgresql/data/]
    environment:
      POSTGRES_DB: hmsdb
      POSTGRES_USER: hmsuser
      POSTGRES_PASSWORD: ${DB_PASSWORD}

  redis:
    image: redis:7-alpine

  celery:
    build: .
    command: celery -A hms_project worker -l info
    depends_on: [redis, db]
    env_file: .env

  celery_beat:
    build: .
    command: celery -A hms_project beat -l info

volumes:
  postgres_data:
```

DAY 29–30 — Cloud Deployment

Deploy to a real server so you can demo with a live URL. This is what separates real projects from localhost-only projects.

Option	Cost	Difficulty	Best For
Railway.app	Free	Easy	Quickest deploy — connects to GitHub, auto-deploy
Render.com	Free	Easy	Free PostgreSQL + Redis + Web service
DigitalOcean	$6/mo	Medium	Most professional, full control
Heroku	$7/mo	Easy	Easy but paid
VPS (local)	Free	Hard	Home server — not accessible publicly

RECOMMENDATION: Use Railway.app or Render.com for Day 29. Free tier, connects directly to GitHub, auto-deploys on git push. Takes 2 hours.

```
# Render.com deployment steps:
# 1. Push code to GitHub
# 2. Go to render.com → New Web Service → Connect GitHub repo
# 3. Add environment variables
# 4. Add PostgreSQL database (free)
# 5. Add Redis (free)
# 6. Deploy!

# requirements.txt must include:
gunicorn
whitenoise  # For static files
psycopg2-binary
dj-database-url
```

Week 4 + Buffer Deliverable

- ■ Admin analytics dashboard with revenue charts
- ■ Real-time notification bell via WebSockets
- ■ All pages polished and mobile-friendly
- ■ Custom error pages
- ■ Docker setup working locally
- ■ Live URL on Railway/Render
- ■ Comprehensive README with screenshots on GitHub

DAY-BY-DAY CHECKLIST

Day	Task	Week	Hours	Done?
Day 1	PostgreSQL migration	Week 1	4h	[ ]
Day 2	Redis + Celery setup	Week 1	5h	[ ]
Day 3	Audit trail system	Week 1	5h	[ ]
Day 4	2FA for doctors/admins	Week 1	5h	[ ]
Day 5	Rate limiting + static folder fix	Week 1	3h	[ ]
Day 6	Reviews app	Week 1	4h	[ ]
Day 7	Dashboard fixes + Week 1 testing	Week 1	4h	[ ]
Day 8	Drug interaction DB + model	Week 2	5h	[ ]
Day 9	Drug checker API + prescription UI	Week 2	5h	[ ]
Day 10	No-show ML — train model	Week 2	6h	[ ]
Day 11	No-show — integrate into booking	Week 2	4h	[ ]
Day 12	Heart attack predictor — train	Week 2	5h	[ ]
Day 13	Heart predictor — form + template	Week 2	5h	[ ]
Day 14	Week 2 testing + bug fixes	Week 2	4h	[ ]
Day 15	AI triage — model + Claude API	Week 3	5h	[ ]
Day 16	AI triage — form + queue view	Week 3	5h	[ ]
Day 17	WhatsApp — Twilio setup + basic	Week 3	4h	[ ]
Day 18	WhatsApp — all events + Celery Beat	Week 3	5h	[ ]
Day 19	AI report reader — Claude + model	Week 3	5h	[ ]
Day 20	AI report reader — template + test	Week 3	4h	[ ]
Day 21	Week 3 testing + integration	Week 3	4h	[ ]
Day 22	Admin reports — data queries	Week 4	5h	[ ]
Day 23	Admin reports — charts + PDF export	Week 4	4h	[ ]
Day 24	WebSockets — Django Channels setup	Week 4	5h	[ ]
Day 25	WebSockets — live notifications	Week 4	5h	[ ]
Day 26	Full polish pass — part 1	Week 4	6h	[ ]
Day 27	Full polish pass — part 2 + README	Week 4	6h	[ ]
Day 28	Docker setup	Week 4	4h	[ ]
Day 29	Cloud deployment	Buffer	5h	[ ]
Day 30	Final testing + demo prep	Buffer	4h	[ ]

ALL NEW MODELS — WEEK 2, 3, 4

App	Model	Key Fields	Week
audit	AuditLog	user, action, model_name, old_values, ip_address	1
reviews	Review	patient, doctor, appointment, rating, comment	1
drug_checker	DrugInteraction	drug_a, drug_b, severity, description, management	2
drug_checker	DrugAllergyAlert	patient, drug, reaction, severity	2
drug_checker	PrescriptionInteractionLog	prescription, drug_a, drug_b, acknowledged	2
ml_noshow	NoShowPrediction	appointment, probability, risk_level, actual_shown	2
ml_noshow	NoShowMLModel	version, accuracy, f1_score, model_file	2
ml_heart	HeartRiskAssessment	patient, 13 UCI features, risk_score, risk_level	2
triage	TriageAssessment	patient, priority P1-P4, symptoms, ai_reasoning	3
triage	SymptomDatabase	name, keywords[], urgency_weight, red_flag	3
ai_reader	ReportAnalysis	user, file, report_type, ai_summary, urgency_flag	3

COMPLETE TECH STACK — ALL 30 DAYS

Category	Technology	When Added	Install Command
DB	PostgreSQL 15	Day 1	pip install psycopg2-binary
Cache	Redis 7	Day 2	pip install redis django-redis
Queue	Celery 5	Day 2	pip install celery django-celery-beat
Security	django-axes	Day 5	pip install django-axes
Security	django-ratelimit	Day 5	pip install django-ratelimit
Security	django-otp	Day 4	pip install django-otp qrcode
ML	scikit-learn	Day 10	pip install scikit-learn
ML	XGBoost	Day 10	pip install xgboost
ML	joblib	Day 10	pip install joblib
ML	pandas + numpy	Day 10	pip install pandas numpy
NLP	RapidFuzz	Day 8	pip install rapidfuzz
AI	Anthropic Claude	Day 15	pip install anthropic
WhatsApp	Twilio	Day 17	pip install twilio
Real-time	Django Channels	Day 24	pip install channels channels-redis daphne
Deploy	Gunicorn	Day 28	pip install gunicorn
Deploy	Whitenoise	Day 28	pip install whitenoise
Deploy	Docker	Day 28	Download Docker Desktop

WHAT YOU HAVE AT END OF 30 DAYS

Here is what your HMS will look like after 30 days compared to a typical student project:

Feature	Typical Student HMS	Your HMS After 30 Days
Database	SQLite	PostgreSQL with connection pooling
Background Tasks	Everything blocking	Celery async — instant response
Security	Basic login	2FA + Rate limiting + Audit trail
Notifications	In-app only	In-app + Email + WhatsApp
AI Features	None	Triage + Drug Checker + No-Show ML + Report Reader
ML Models	None	Heart predictor + No-show predictor
Drug Safety	None	200k+ interaction checks
Real-time	None / 30s polling	WebSocket live updates
Patient Communication	Email only	WhatsApp + Email + In-app
Reports	None	Admin analytics + Chart.js + PDF export
Deployment	localhost only	Live cloud URL (Railway/Render)
Code Quality	Monolith	Services, tasks, signals, clean architecture
Portfolio Value	Basic	Senior-level, production-grade

■ YOUR UNIQUE VALUE PROPOSITION AFTER 30 DAYS

"The only HMS that predicts no-shows, checks drug interactions in real-time, triages patients with AI, and communicates via WhatsApp — built specifically for South Asian hospitals where 80% of patients use WhatsApp daily."

This is not a student project anymore. This is a real product.

Build something that saves lives. You can do this. ■